Innominatum CONSULTORIA

Programação em JavaScript

ServiceNow - Deloitte

Rodrigo Costa

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Innominatum CONSULTORIA

Fundamentos Angular

Módulo 5

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Comunicação entre Componentes & Ciclo de Vida

Sessão 27

Objetivo geral

Capacitar os formandos na construção de arquiteturas baseadas em múltiplos componentes, compreendendo os mecanismos de comunicação entre componentes pai e filho através dos decorators **@Input** e **@Output**, bem como o ciclo de vida dos componentes Angular. A sessão estabelece a base para a criação de interfaces modulares e reativas, permitindo controlar o fluxo de dados e executar lógica em momentos específicos da existência do componente.

Objetivos específicos

Ao final da sessão, os formandos deverão ser capazes de:

- Transmitir os dados de componentes pai para componentes filho, utilizando o decorator **@Input**;
- Emitir eventos de componentes filho para componentes pai, utilizando o decorator **@Output** e **EventEmitter**;
- Compreender o fluxo de comunicação unidirecional do Angular, respeitando a arquitetura baseada em componentes;
- Implementar hooks do ciclo de vida, como **OnInit** e **OnDestroy**;
- Executar lógica em momentos específicos do ciclo de vida, garantindo controlo e previsibilidade do comportamento do componente.

Comunicação Pai -> Filho (@input)

Arquitetura baseada em componentes

Em Angular, componentes formam uma árvore hierárquica, na estrutura que criamos até agora temos apenas dois componentes que se comportam como pai e filho, onde o App é o componente pai em relação ao Header, que logo é o componente filho.

```
src > app > app.html > ...
1   <app-header></app-header>
2   <router-outlet/>
```

Ao utilizar o seletor <app-header> dentro do template do App, estabelecemos uma relação hierárquica onde o App é o componente pai e o Header é o componente filho.

O decorator @Input

Para utilizarmos o @Input devemos importar no componente filho. Nesse caso, vamos importar o Input no header.ts

```
2   import { Component, OnInit, OnDestroy, Input } from '@angular/core';
```

No componente filho (header), utilizamos o decorator @Input() para permitir que o componente pai envie dados para ele.

```
15  export class Header implements OnInit, OnDestroy {
16    @Input() titulo!: string;
```

No app.html iremos utilizar o **property binding** [titulo] no elemento app-header passando o título que desejamos exibir na tela.

```
<app-header [titulo]="'Titulo criado no App'"></app-header>
```

Comunicação Filho -> Pai (@Output)

O decorator @Output - header

Innominatum CONSULTORIA

Agora precisamos importar o Output e o EventEmmiter no nosso componente Header

```
import { Component, OnInit, OnDestroy, Input, Output, EventEmitter } from '@angular/core';
```

utilizamos o decorator @Output() para permitir que o componente filho emita eventos que o componente pai poderá escutar.

```
15  export class Header implements OnInit, OnDestroy {
16    @Input() titulo!: string;
17    @Output() logout = new EventEmitter<void>();
```

No header.ts iremos utilizar o método logout que irá emitir o evento para o componente pai (app.ts).

```
38  onLogout(){
39    this.logout.emit();
40  }
```

O decorator @Output – app

No app.html iremos utilizar o **event binding** (logout) no elemento app-header passando o método fazerLogout() como valor.

```
<app-header [titulo]="'Titulo criado no App'" (logout)="fazerLogout()"></app-header>
```

No app.ts iremos criar o método fazerLogout, que irá ser acionado quando o evento acontecer no header (elemento filho), e iremos criar uma lógica a ser executada, no momento que esse evento acontecer.

```
13    fazerLogout() {
14      alert('Utilizador fez logout!');
15    }
16  }
```

O @Output envia dados no evento – Header

No componente header criamos agora um @Output tipado com string.

```
@Output() utilizadorSelecionado = new EventEmitter<string>();
```

Criamos o método selecionar, no header.ts

```
selecionar() {
  this.utilizadorSelecionado.emit("Ana");
}
```

No header.html criamos um botão com o método selecionar.

```
<button (click)="selecionar()">Selecionar Utilizador</button>
```

O @Output envia dados no evento – App

No app.ts criamos o método receber que espera uma string como parâmetro.

```
receber(nome: string) {
  alert('Utilizador selecionado: ' + nome);
}
```

No app.html utilizamos Event Binding para escutar o evento utilizadorSelecionado e passamos o valor recebido ($event) para o método receber.

```
<app-header [titulo]="'Titulo criado no App'" (logout)="fazerLogout()"
(utilizadorSelecionado)="receber($event)"></app-header>
```

Ciclo de Vida dos Componentes

O que é ciclo de vida (Lifecycle Hooks)?

Todo componente Angular passa por diferentes fases desde sua criação até a sua destruição. O Angular disponibiliza métodos especiais chamados hooks que permitem executar código em cada uma dessas fases.

Hook OnInit

É executado após o Angular inicializar as propriedades do componente, ele é muito utilizado para:

- Carregar dados;
- Inicializar estado;
- Fazer chamadas HTTP.

```
import { Component, OnInit } from '@angular/core';
```

```
ngOnInit(): void {
  console.log('Header component inicializado');
}
```

Hook OnDestroy

É executado quando o componente é destruído, ele será usado para:

- Limpar subscrições;
- Limpar timers;
- Libertar recursos.

```
import { Component, OnInit, OnDestroy } from '@angular/core';
```

```
export class Header implements OnInit, OnDestroy {
```

```
ngOnDestroy(): void {
  console.log('Header component destruído');
}
```

Síntese

- **Comunicação Pai -> Filho (@Input)**: Utilização do decorator **@Input** para permitir que componentes pai transmitam dados para componentes filho através de property binding;
- **Comunicação Filho -> Pai (@Output e EventEmitter)**: Implementação do decorator **@Output** para permitir que componentes filho emitam eventos para os seus componentes pai;
- **Fluxo de dados controlado e declarativo**: Compreensão do modelo arquitetural do Angular, onde dados fluem de cima para baixo (**@Input**) e eventos fluem de baixo para cima (**@Output**);
- **Ciclo de vida dos componentes**: Introdução aos hooks OnInit e OnDestroy, permitindo executar lógica no momento apropriado da existência do componente, como inicialização de dados e limpeza de recursos.

Conclusão

Innominatum CONSULTORIA

Programação em JavaScript

ServiceNow - Deloitte

Rodrigo Costa

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL